```
  </body>
</html>
EOS;

$dom = Dom\HTMLDocument::createFromString($html5);
var_dump($dom->querySelectorAll('body p'));
var_dump($dom->querySelector('body p'));
var_dump($dom->querySelector('body p')->textContent);

Output:
object(Dom\NodeList)#5 (1) {
  ["length"]=>
  int(3)
}
object(Dom\HTMLElement)#5 (29) {
  ["namespaceURI"]=>
  string(28) "http://www.w3.org/1999/xhtml"
  ...
}
string(8) "Para One"
```

The above shows the main difference between `querySelector()`, which always returns a single node, and `querySelectorAll()`, which always returns a node list. This example also shows that `querySelector()` returns the first match, as we can deduce when accessing the returned node's `textContent` property. The list returned by `querySelectorAll()` is said to be "static" (which matches the behavior of JavaScript's equivalent `document.querySelectorAll()`), static node lists do not update if there are changes to the DOM structure after selection. If we obtained a list of `p` element children of `main` and then added another `p` element, we would not see the new element in the list (the query would have to run again to obtain a new list). A `querySelectorAll()` node list contains references to the matching DOM nodes, so any changes to those nodes' attributes, contents, or child elements would be accessible without needing to query again.